Crowd Counting on Edge Devices

DA642E Project

Adrian Jon Berlovan
Athanasios Smilkos
Vrysiida Charizi

Computer Science: Internet of Things
Malmö University
Sweden
March 19, 2026

Abstract

In the current day and age, our right to privacy is being infringed more and more often. The goal of this project is to help people regain that right, especially in public settings. Often times cameras located in public areas need to send data back to the cloud in order to do inference or segmentation, not caring about privacy. In order to do image segmentation in a privacy-friendly way, we will be investigating running such machine learning (ML) algorithms directly on edge devices. This subject has been often tackled in literature, however usually using some form of hybrid edge, server and federated learning.

We have combined several previously existing datasets into a new one, with a smaller selection of persons per image, which is more representative of small gatherings in public places where people might be easily identifiable. The *YOLO26* (You Only Look Once, version 26) pre-trained neural network has been re-trained on this dataset and we have employed quantization in order to be able to run the model faster on lower end hardware.

1 Introduction

Inspired by the many privacy violations we see and hear about almost daily, we have decided to investigate the feasibility of a segmentation model running on edge devices. Since one of the places that these violations happen most often is public spaces, we have synthesized our own dataset by combining existing, open-source datasets that are frequently featured in the literature. We have taken samples from *COCOPersons (Common Objects In Context), CrowdHuman* and *KITTI*. The selected samples (that fit our criteria) were added to a dataset that was subsequently used to re-train a pre-trained YOLO[7] model, specifically version 26[10].

2 Related Work

The model we are basing our project on (*YOLO*) comes in various sizes. Since our intention is to deploy to edge devices, we have chosen the smallest model size, *YOLO26n*, which has 2.4 milion parameters. However, the model is available in other variants as well, the largest being *YOLO26x* at 55.7 milion parameters. Other work has been done in this domain[8][2], however the models are usually not released, and as such the results are tougher to replicate and verify. Many of the related research does crowd counting without segmentation[5][1], which is good for very large gatherings but might prove less applicable to smaller groups of people.

Many deep learning models are too computationally expensive for deployment on resource-constrained hardware. Other research proposes[4] a method based on feature fusion, Bayesian loss, and has reported accuracy close to state-of-the-art methods. It also support real-time inference on edge devices.

Other studies[5] have investigated crowd counting on resource-constrained edge devices. They noted that a lot of existing methods focus on accuracy and overlook model size and cost. They propose a weighted knowledge distillation method to improve lightweight crowd counting models and evaluate it on five benchmark datasets.

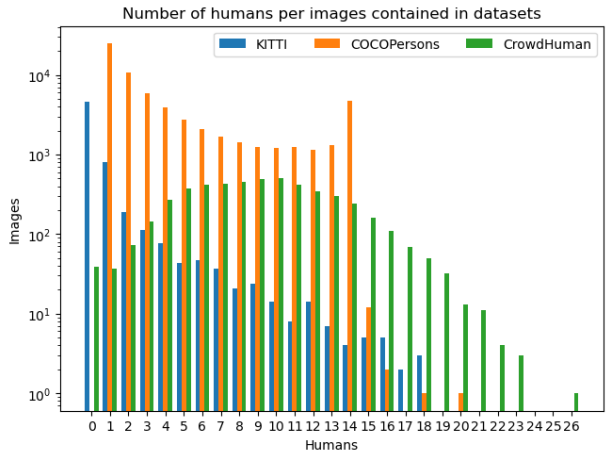


Figure 1: Composition of different datasets before filtering.

3 Method

The principal goal of this project centers on achieving high-accuracy image segmentation on resource-constrained hardware while maintaining data privacy. Our approach is divided into three phases:

- Dataset synthesis - filtering and combining existing datasets
- Model architecture selection - explore suitable segmentation models, preferably pre-trained
- Optimization for edge deployment - quantization

To better reflect the privacy challenges of small-scale public interactions, we developed a composite

dataset by merging several existing open-source computer vision libraries, specifically COCO[3], KITTI[6] and CrowdHuman[9] datasets. Unlike standard datasets that often feature crowded urban scenes, our selection was filtered to prioritize images with a lower density of persons. This specific focus ensures the model is trained to recognize individuals in small gathering scenarios - for example, the scenario that we mentioned in our project proposal (a monitoring camera in a bus, for statistics on tickets used), as a main application area. In this case, the risk of identification is highest and the need for privacy-preserving segmentation is most acute.

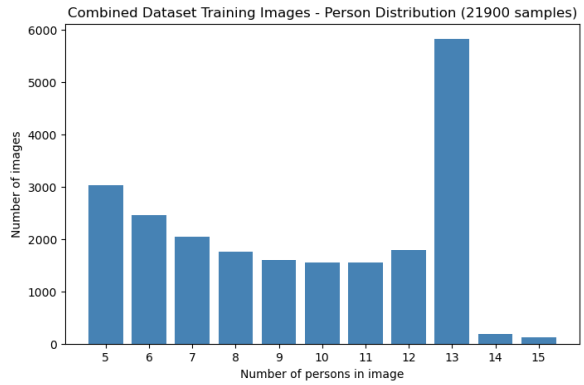
Unfortunately, many of the related research in running crowd counting and object segmentation on edge did not disclose their pre-trained models. For our model architecture, we decided to use the latest version of the *YOLO* architecture. It is used specifically for real-time object detection and image segmentations in various deep learning applications[10]. The network was then re-trained with our custom-curated dataset so that the model can better learn what people look like in public spaces, with a specific focus on their shapes and sizes in small groups.

Finally, in order to bring this model to lower-end hardware, we implemented quantization. This process involves converting the model's weights and biases from high precision floating-point formats (e.g. FP32 - floating point on 32 bits) to lower-bit formats (e.g. FP16 - half-precision floats at 16 bits or even INT8). This results in significantly reduced computational overhead on the CPU/GPU of the edge device and also the model's storage footprint. With this method, we have reduced the model size by half.

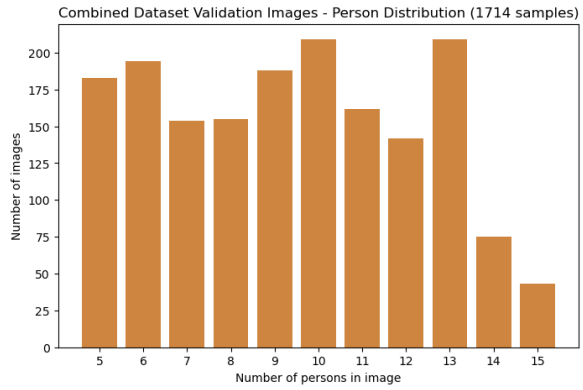
4 Results

The datasets we have selected contained a large amount of various images featuring diverse scenes. For the scope of our project, we are only interested in images that contain between **5-15** persons, as this is a good representation of small groups of people in public spaces. Looping through labels, everything that is not a label for a person was discarded. If the number of labels left over was in the above range, the image was kept for the custom combined dataset. This has helped us reduce the size of the dataset a lot, by filtering out images that were not relevant. In Figure 1, we can observe the original dataset composition. Please note that the graph *y axis* is in logarithmic scale, as some datasets had a lot more data compared to others.

After the datasets have been processed and the relevant images extracted, we have compiled our dataset. The resulting distribution can be seen in 2. An interesting feature to be observed is the spike that occurs on 13 people per image. So far, science has been unable to explain it.



(a) Training

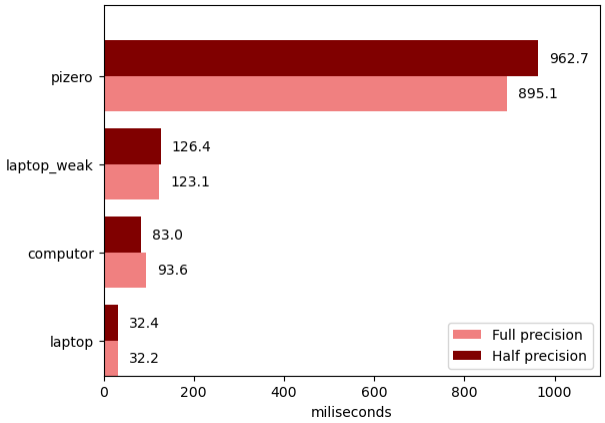


(b) Validation

Figure 2: Combined dataset for training and validation.

With the reduced dataset size, the model was trained on a desktop computer with the following specifications: *Ryzen 5 2600X CPU, 16GB RAM, Nvidia 1660 6GB GPU*. The model was trained for 25 epochs, with the results available in Appendix B. The original model as well as our re-trained model have been benchmarked using COCO8, a subset of COCO used for experimentation and testing. In it, our model has performed better across the board when compared to the original YOLO26n. We have achieved smaller model size, faster inference and more frames per second (FPS). The only worse-performing metric is accuracy, due to our model only being trained on one label (person) versus the original 80 that *YOLO* is capable of. The results can be observed in Appendix A.

Aditionally, we have quantized the model and ran it on various hardware with limited speeds. For benchmarking, we used 23 never-seen-before (by the model) images, and ran inference on them 5 times. The results reported in 3 represent the average inference speed across 115 inferences. The results are reassuring - even an extremely cheap and limited hardware such as *Raspberry Pi Zero 2W* is able to run our model at reasonable speeds. If sporadic snapshots are enough, this device is capable of running such a workload.



pizero: ARM Cortex-A53 1GHz
laptop weak: Intel Core i5 6200U 2.3 GHz, iGPU Intel HD 520
computor: AMD Ryzen 5 2600X 3.6 GHz
laptop: AMD Ryzen 7 7730U 2.0 GHz, iGPU Radeon Vega 8

Figure 3: Average inference time in miliseconds.

We have attempted to configure and run the model on an original ESP32 as well. However, given that it is very old and dated hardware, we have been unable to get it working. Some of the models that do run on that hardware are fairly dated themselves, and have already been thoroughly investigated.

5 Conclusions

In conclusion, this project successfully demonstrated the feasibility of deploying a privacy-preserving image segmentation model on resource-constrained hardware. By synthesizing a custom dataset which was filtered specifically for small gathering scenarios (5-15 people). We optimized the *YOLO26n* architecture to perform effectively in scenarios where individual privacy is most at risk. Through the application of quantization, we reduced the model's memory footprint and increased inference speed, outperforming the original *YOLO26n* benchmarks.

While the results we achieved after only 25 training epochs looked encouraging, we hope that in the future we will be able to increase it to at least 100 epochs, which we believe will lead to further improvements. For future work, and after thoroughly researching the ESP32-based board with its significant limitations, we believe that the model can be reduced further and potentially even be able to run on such boards, with the use of INT8 quantization.

Given the promising results we have achieved, the model has a strong potential for further improvement. With little work, an open source model such as YOLO26 can be re-trained and quantized to run reliably on inexpensive and slow devices, eliminating the need for cloud processing. Ultimately, this work proves that high-accuracy segmentation does not have to come at the cost of data privacy or inexpensive and unreliable cloud infrastructure.

References

[1] Mas Nurul Achmadiah, Chi-Chia Sun, Wen-Kai Kuo, and Jun-Wei Hsieh. Repsfnet : A single fusion network with structural reparameterization for crowd counting. In *2025 IEEE International Conference on Advanced Visual and Signal-Based Systems (AVSS)*, page 1–6. IEEE, August 2025.

[2] Jiwei Chen, Qi Wang, Junyu Gao, Jing Zhang, Dingyi Li, and Jing-Jia Luo. One-shot crowd counting with density guidance for scene adaptaion, 2026.

[3] COCO. Common objects in context, 2026. Accessed: Feb. 15, 2026.

[4] Zuo Huang, Richard Sinnott, and Qiuhong Ke. Crowd counting using deep learning in edge devices. In *Proceedings of the 2021 IEEE/ACM 8th International Conference on Big Data Computing, Applications and Technologies*, BDCAT '21, page 28–37, New York, NY, USA, 2022. Association for Computing Machinery.

[5] Muhammad Asif Khan, Hamid Menouar, Ridha Hamila, and Adnan Abu-Dayya. Crowd counting at the edge using weighted knowledge distillation. *Scientific Reports*, 15(1):11932, Apr 2025.

[6] KITTI. Kitti vision benchmark suite, 2026. Accessed: Feb. 15, 2026.

[7] Joseph Redmon, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. *CoRR*, abs/1506.02640, 2015.

[8] Tomás Mestre dos Santos, Rui Neto Marinheiro, and Fernando Brito e Abreu. Wireless crowd detection for smart overtourism mitigation. In *Smart Life and Smart Life Engineering: Current State and Future Vision*, pages 237–258. Springer, 2025.

[9] Shuai Shao, Zijian Zhao, Boxun Li, Tete Xiao, Gang Yu, Xiangyu Zhang, and Jian Sun. Crowdhuman: A benchmark for detecting human in a crowd. *CoRR*, abs/1805.00123, 2018.

[10] Ultralytics. Yolo26, Accessed: Mar. 18, 2026. https://docs.ultralytics.com/models/yolo26/.

A Appendix

#	Format	Status	Size (MB)	mAP50-95(B)	Inference (ms/im)	FPS
1	PyTorch	✓	5.1	0.078	171.39	5.83
2	TorchScript	✓	9.7	0.0749	284.01	3.52
3	ONNX	✓	9.4	0.0749	154.91	6.46
4	OpenVINO	✓	9.5	0.0749	222.11	4.50
5	TensorRT	×	0.0	-	-	-
6	CoreML	⊗	4.8	-	-	-
7	TensorFlow SavedModel	✓	23.5	0.0749	359.22	2.78
8	TensorFlow GraphDef	✓	9.4	0.0749	357.28	2.80
9	TensorFlow Lite	✓	9.4	0.0749	139.26	7.18
10	TensorFlow Edge TPU	⊗	3.9	-	-	-
11	TensorFlow.js	⊗	9.7	-	-	-
12	PaddlePaddle	✓	18.7	0.0726	308.83	3.24
13	MNN	✓	9.2	0.0749	155.57	6.43
14	NCNN	✓	9.3	0.0726	201.40	4.97
15	IMX	×	0.0	-	-	-
16	RKNN	×	0.0	-	-	-
17	ExecuTorch	×	0.0	-	-	-
18	Axelera	×	0.0	-	-	-

Benchmark legend: ✓ Success ⊗ Export passed but validation failed × Export failed

Table 1: Custom dataset trained YOLO26n benchmark

#	Format	Status	Size (MB)	mAP50-95(B)	Inference (ms/im)	FPS
1	PyTorch	✓	5.3	0.6601	182.42	5.48
2	TorchScript	✓	9.8	0.6881	331.02	3.02
3	ONNX	✓	9.5	0.6881	154.72	6.46
4	OpenVINO	✓	9.6	0.6881	148.79	6.72
5	TensorRT	×	0.0	-	-	-
6	CoreML	⊗	4.8	-	-	-
7	TensorFlow SavedModel	✓	24.6	0.6881	372.79	2.68
8	TensorFlow GraphDef	✓	9.5	0.6881	623.82	1.60
9	TensorFlow Lite	✓	9.9	0.6881	164.06	6.10
10	TensorFlow Edge TPU	⊗	3.9	-	-	-
11	TensorFlow.js	⊗	10.0	-	-	-
12	PaddlePaddle	✓	18.9	0.671	340.47	2.94
13	MNN	✓	9.4	0.6881	186.97	5.35
14	NCNN	✓	9.4	0.671	251.53	3.98
15	IMX	×	0.0	-	-	-
16	RKNN	×	0.0	-	-	-
17	ExecuTorch	×	0.0	-	-	-
18	Axelera	×	0.0	-	-	-

Benchmark legend: ✓ Success ⊗ Export passed but validation failed × Export failed

Table 2: YOLO26n model benchmark

B Appendix

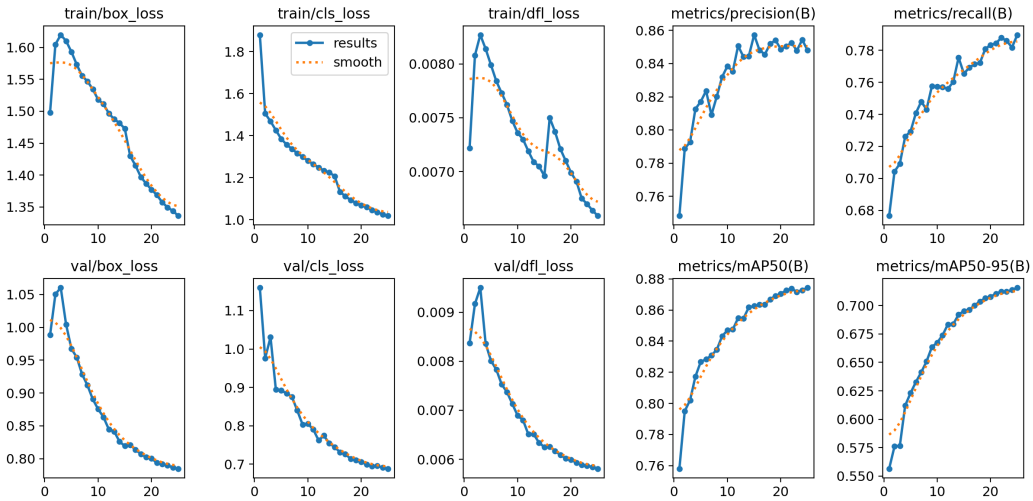


Figure 4: Training results

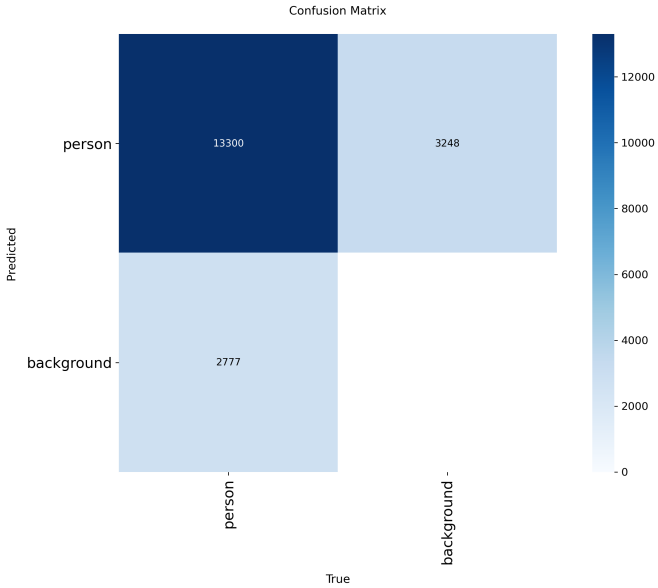


Figure 5: Confusion matrix